```
. . Created /home/gene/clojure/incanter_demo/incanter_demo-1.0.0-
SNAPSHOT-standalone.jar
```

Now that you have the standalone JAR for your project, run it like a typical Java JAR:

$java -jar incanter_demo-1.0.0-SNAPSHOT-standalone.jar. You should see the following output:

```
Welcome to Incanter demo
```

You should also see the plot, as shown in Figure 1. Now, if you go back to the project directory and list the contents of the *lib* sub-directory, you will see a bunch of JARs—which are basically the dependencies you specified in the *project.clj* file.

It is to be noted here that the option *uberjar* tells *lein* to build a standalone JAR for your application, so as to make it shareable. If, however, you just want to run your application, use *lein run*.

Having had a preliminary look at Clojure, the language, let us now learn a bit about using Clojure for Web applications.

Web frameworks for Clojure

While there is nothing to prevent you from using the Java Servlet API directly in your Clojure applications and deploying them as Web Archives (WARs), we will not do that today. Instead, we will take a brief look at one of the Clojure Web frameworks, *Conjure (https://github.com/macourtney/Conjure)*. As the website claims, it's a Rails-like framework for Clojure. How similar the two are can only be judged by a Rails aficionado, which I am not. So let's use Leiningen to write a simple Web application using Conjure.

Create a new Clojure project; add *conjure-core* as one of the dependencies, and *lein-conjure* in the *dev-dependencies* list. The *project.clj* should look like what's shown below:

```
(defproject hello-conj "1.0.0-SNAPSHOT"
  :description "Hello Conjure!"
  :dependencies [[org.clojure/clojure "1.2.0"]
                 [org.clojure/clojure-contrib "1.2.0"]
       [conjure-core "0.8.0"]]
  :dev-dependencies [[lein-conjure "0.8.0"]])
```

Then execute *lein deps* to download all the dependencies, which will be stored in the *lib/* directories. Now that the *conjure* plug-in for Leiningen has been downloaded, let us convert this generic Clojure project into a Conjure project:

```
$ lein conjure new
```

This step will create a number of Conjure-specific sub-directories and files in the project folder, which you can view by using the tree command. Let us now test the working of our server: *lein conjure server.* You should see something like what is given below:

```
DEBUG [conjure.core.db.flavors.h2]: Executing query: ["SELECT *
FROM sessions LIMIT 1"]
DEBUG [conjure.core.db.flavors.h2]: Create table: :sessions with
specs: (("id" "INT" "NOT NULL" "AUTO_INCREMENT" "PRIMARY KEY")
["created_at" "TIMESTAMP"] ["session_id" "VARCHAR(255)") ["data"
"TEXT"])
INFO  [conjure.core.server.server]: Server Initialized.
INFO  [conjure.core.server.server]: Initializing plugins...
INFO  [conjure.core.server.server]: Plugins initialized.
INFO  [conjure.core.server.server]: Initializing app
controller...
INFO  [conjure.core.server.server]: App controller initialized.
2011-02-13 16:09:37.909::INFO:  Logging to STDERR via org.
mortbay.log.StdErrLog
2011-02-13 16:09:37.911::INFO:  jetty-6.1.14
2011-02-13 16:09:37.946::INFO:  Started
SocketConnector@0.0.0.0:8080
.
```

. Now, you can visit the URL *http://127.0.0.1:8080* and you should see a default Conjure home-page, as shown in Figure 2.

Congratulations! You can change the default file, which is in *src/view/home/index.clj*, and see the changes automatically; no server restart is required.

Where next?

In this admittedly whirlwind tour of Clojure, we have taken a look at some of the basic features of this relatively new language, and have become somewhat familiar with its ecosystem. The next steps will have to be taken based on what your area of interest is. Accessing SQL and NoSQL databases and multi-threaded applications (which, incidentally, happen to be one of Clojure's selling points) are a couple of areas which can be immediately looked into, especially for building scalable Web applications. Before we part, here's something the Android fans among you should love. There is a Clojure REPL application available in the Android market place, which you can install for free and program in Clojure on the go! END

Resources

1. Clojure - Functional Programming for the JVM: *http://java.ociweb.com/mark/clojure/article.html*
2. Programming Clojure, Stuart Halloway (The Pragmatic Bookshelf)
3. Clojure docs: *http://clojuredocs.org/*
4. Notes on Leiningen: *http://asymmetrical-view.com/2010/06/03/leiningen.html*

By: Amit Saha

The author blogs at *http://amitksaha.wordpress.com* and welcomes queries and suggestions at *amitsaha.in@gmail.com*.